

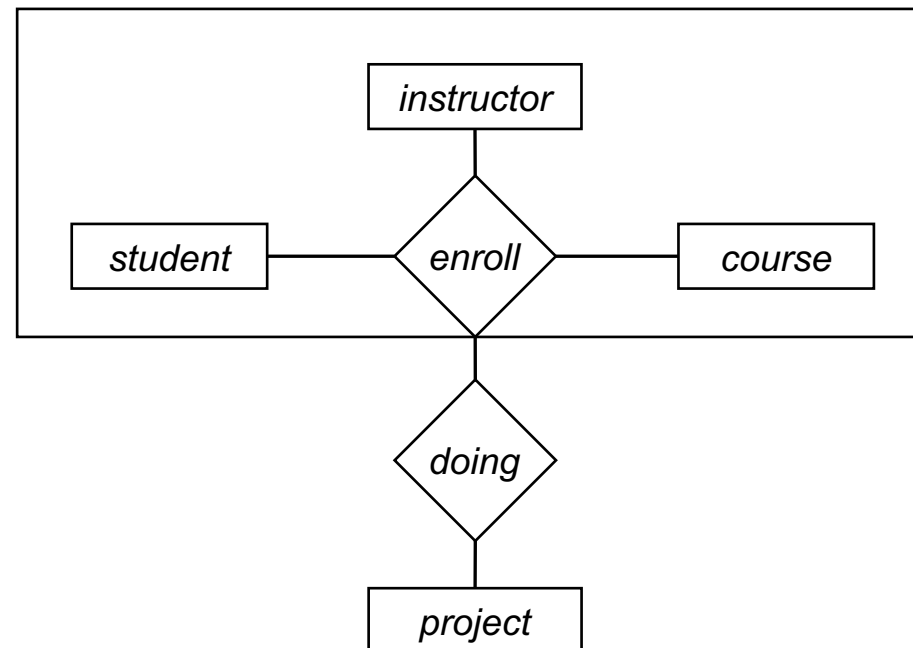
Tutorial 03

Outline

- Aggregation
 - Aggregation is an abstraction through which we can represent relationships as higher level entity sets.
- Weak Entity Sets
- Generalization and Specialization

Aggregation

- A better model is let “student”, “instructor”, “course”, and “enroll” form an abstract entity as **an aggregation**.
- Then “project” only need to associate with the courses which has a course project.

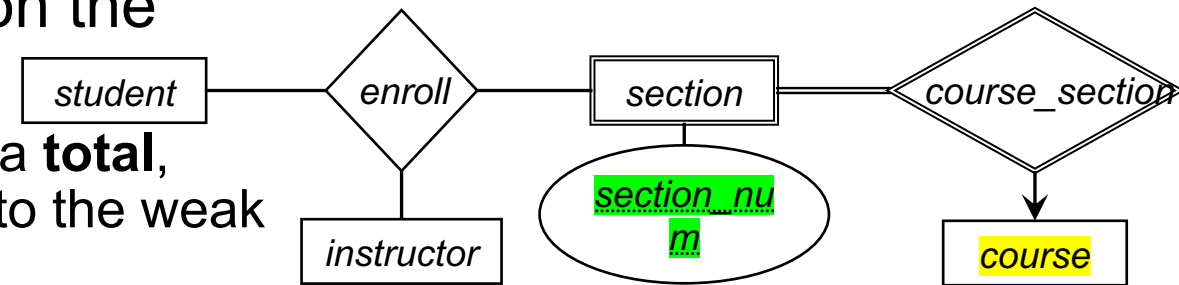


Weak Entity Sets

- A **weak entity** is an entity cannot exist alone. The existence depends on another entity of a different type.
- A set of weak entities is a **weak entity set**.
- Since a weak entity cannot exist alone, it **does not have a key**.
- The existence of a weak entity set depends on the existence of an **identifying entity set**.
 - A weak must relate to the identifying entity set via a **total, one-to-many** relationship set from the identifying to the weak entity set.
 - An **identifying relationship** depicted using a double diamond.
- The **discriminator** (*or partial key*) of a weak entity set is the set of attributes that distinguishes among all the entities of a weak entity set.
- To distinguish weak entities, one must combine **the key of the identifying entity set and the discriminator**.

Weak Entity Sets

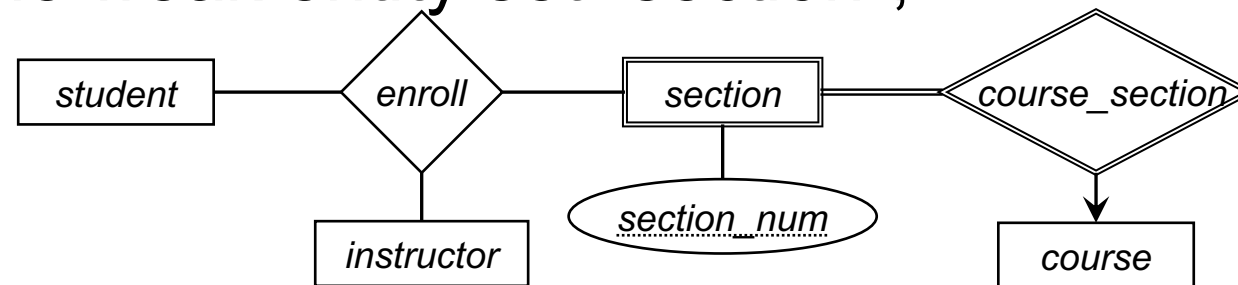
- The existence of a weak entity set depends on the existence of an **identifying entity set**.
 - A weak must relate to the identifying entity set via a **total, one-to-many** relationship set from the identifying to the weak entity set.
 - An **identifying relationship** depicted using a double diamond.
- The **discriminator** (or *partial key*) of a weak entity set is the set of attributes that distinguishes among all the entities of a weak entity set.
- To **distinguish** weak entities, one must combine **the key of the identifying entity set and the discriminator**.



COMP 3013
section 1004

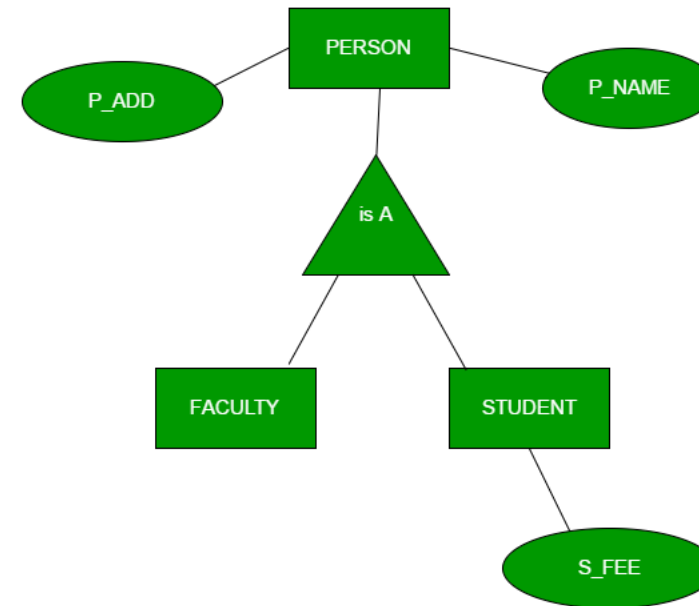
Weak Entity Sets

- **Double rectangles** denote weak entity sets.
- **Dashed underlines** denote discriminators of weak entity sets.
- **Double diamonds** denote identifying relationship sets.
- Assume we model the original “enroll” relationship set without constraints.
- After adding the weak entity set “section”,



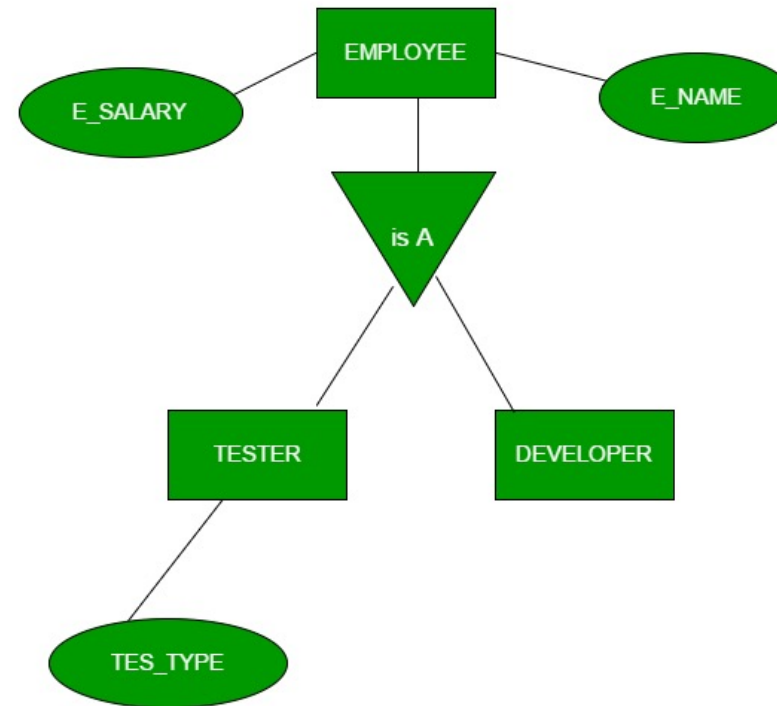
Generalization and Specialization

- Generalization is the process of extracting common properties from a set of entities and create a generalized entity from it. It is a **bottom-up** approach in which two or more entities can be generalized to a higher level entity if they have some attributes in common.



Generalization and Specialization

- In specialization, an entity is divided into sub-entities based on their characteristics. It is a **top-down** approach where higher level entity is specialized into two or more lower level entities.



Specialization

Constraints on Generalization/Specialization

- Cardinality constraints and participation constraints can be applied on ISA relationship sets.
- But some constraints are omitted because
 - each entity on the lower-level is one entity on the higher-level (the entity sets on the lower-level always fully participate ISA);
 - one entity cannot be associated with multiple entities in the same entity set on the lower-level.
- Thus, the constraints on ISA only discuss
- Does one higher-level entity belong to at least one lower-level entity set?
 - If yes, this ISA is a ***total generalization***.
- Does one higher-level entity belong to multiple lower-level entity set?
 - If No, it is a ***disjoint generalization***.

Lab.

Motivation

- Are you tired of condition checking in cross table queries?
- The **NATURAL JOIN** operator associates two tables by the **common attributes**.
- After **NATURAL JOIN**, the duplicated attributes are omitted.

Join Condition

- Join conditions define in which condition the tuples are associated.
- Two tuples are associated if
 - **NATURAL**: all common attributes have the same value.
 - **ON** <predicate>: the predicate is evaluated to be true.
 - **USING** ($A_1, A_2, \dots, A_n$): the common attributes in list have the same value.
- Notes:
 - The predicate in the ON clause is user defined, which is very flexible.
 - NATURAL and USING combine the common attributes. But **ON duplicates common attributes.**

Join Condition

- A JOIN without any condition is same as a cartesian product.

Join Type

- Sometimes users want to keep the unmatched tuples after joining two tables.
- **OUTER JOIN** can handle it.
 - **table1 NATURAL LEFT OUTER JOIN table2**
All tuples in table1 are in the result. For the unmatched tuples, the values of the attributes from table2 are **NULL**, meaning “unknown”. (NULL values will be introduced in following labs.)
 - **table1 NATURAL RIGHT OUTER JOIN table2**
The unmatched tuples from table2 are kept.
 - **table1 NATURAL FULL OUTER JOIN table2**
All tuples (from both table1 and table2) are kept.
- NATURAL is the join condition.
- On the opposite of **OUTER**, **INNER JOIN** does not keep the unmatched tuples.
- Same as JOIN. “INNER” is usually omitted.

1. Find the films (title) played by Zero Cage.

```
SELECT title  
FROM film JOIN film_actor USING (film_id) JOIN actor USING  
(actor_id)  
WHERE first_name='Zero' AND last_name='Cage'
```

Attributes `film_id` and `actor_id` have same value in the joined tables

2. Find the films (title) rented by George Linton. The join condition is **ON**.

```
SELECT title
```

```
FROM customer JOIN rental ON
```

```
customer.customer_id=rental.customer_id JOIN inventory ON
```

```
rental.inventory_id=inventory.inventory_id JOIN film ON
```

```
inventory.film_id=film.film_id
```

```
WHERE first_name='George' AND last_name='Linton'
```


3. Find the customers (name) who have rented some action (category) films. The join condition is **USING**.

```
SELECT first_name, last_name
```

```
FROM customer JOIN rental USING(customer_id) JOIN inventory  
USING(inventory_id) JOIN film_category USING(film_id) JOIN  
category USING(category_id)
```

```
WHERE name='Action'
```

4. Join the tables **film**, **film_category**, and **category**, using both conditions **ON** and **USING**.

```
SELECT *
```

```
FROM film JOIN film_category USING(film_id) JOIN category  
ON(film_category.category_id=category.category_id)
```

5. Find all pairs of customers (name) who have rented a same film. Any join condition is fine.

```
SELECT c1.first_name, c1.last_name, c2.first_name, c2.last_name
FROM inventory AS i1 JOIN inventory AS i2 ON
i1.inventory_id <> i2.inventory_id AND i1.film_id = i2.film_id JOIN
rental AS r1 ON i1.inventory_id = r1.inventory_id JOIN rental AS r2
ON i2.inventory_id = r2.inventory_id JOIN customer AS c1 ON
r1.customer_id = c1.customer_id JOIN customer AS c2 ON
r2.customer_id = c2.customer_id;
```

6. Find the films rented by each customer. If a customer has not rented any film, give it a NULL value.

```
SELECT first_name, last_name, title
```

```
FROM customer LEFT OUTER JOIN rental USING(customer_id) LEFT  
OUTER JOIN inventory USING(inventory_id) LEFT OUTER JOIN film  
USING(film_id)
```